



Where does the time go? Understanding performance using SPACE-Timers

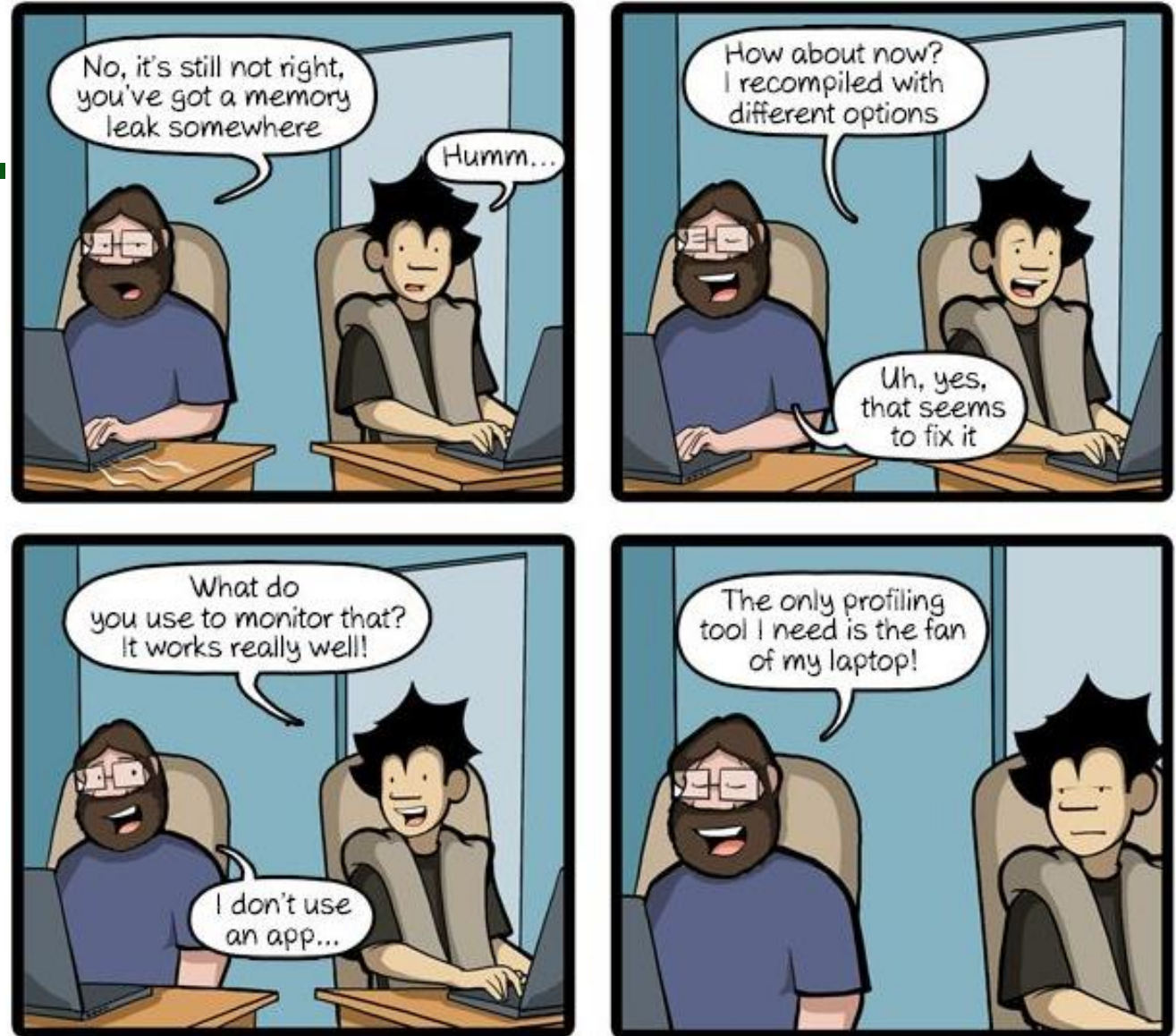
04.05.2026

-

Geray Karademir

What is profiling?

- **profiling** means analysing a program dynamically by measuring, for example, the memory or time usage of a program.
- Today we focus on measuring runtime.



CommitStrip.com

Different levels and ways to profile:

Manual instrumentation

- Manual, unstructured, local measurements
- Tools: `printf`, `std::cout`, `std::chrono`, ...

Classical profilers

Function-level: Measure total time spent in each function by instrumenting function entry/exit

Sampling tools: periodically sample the call stack to estimate where time is spent (e.g. `perf`)

Identify hotspots: Highlight the most time-consuming functions to guide optimization efforts

Tools: `gprof`, `perf`, `gperftools`

Tracing

Record timestamped events across the system

Timeline view of execution (CPU, GPU, threads)

Useful for concurrency and latency analysis

Tools: NVIDIA Nsight Systems, NVTX, ITT, ROCtx, Chrome Tracing, Omnitrace

SPACE-Timers

SPACE-Timers - A Stack-Based Hierarchical Timing
System for C++

Geray S. Karademir^a, Klaus Dolag^{a,b}

^a*University Observatory, Faculty of Physics, Ludwig-Maximilians-Universität
München, Scheinerstraße 1, 81679, Munich, Germany*

^b*Max-Planck-Institut für Astrophysik, Karl-Schwarzschild-Straße 1, 85740, Garching near
Munich, Germany*

A hybrid between manual instrumentation and full tracing tools.

Link:

<https://ui.adsabs.harvard.edu/abs/2026arXiv260301618K/abstract>

Repo:

<https://gitlab.lrz.de/MAGNETICUM/SPACE-Timers>

SPACE-Timers: key capabilities

- SPACE-Timers turns raw timing into a hierarchical, scalable performance model tailored for HPC applications.



**Stack-based
hierarchical
model**



**Recursive
aggregation &
precise
attribution**



**Detection of
unaccounted
time and internal
error detection**



**Compact runtime
summaries**



**Checkpointing
and restart
capabilities**



**Multiple
Backends: NVTX,
ITT, ROCtx,
Omnitrace**



**Integrates with
MERIC for
energy-aware
optimisation**

Repo Examples

Link to examples:

<https://gitlab.lrz.de/MAGNETICUM/SPACE-Timers/-/tree/main/demo>

Application in OpenGadget3

Timer output

TIMER_WRITE_TIMES_TO_CPUFILES();

Writing of “cpu.txt” files storing the results of the main rank

```
Step 625 Time: a=0.323168, MPI-Tasks: 2 Task:0
Total wall clock time for Global = 7326.7 sec
* Timestep : 7325.4034 sec, 99.98%
- * GRAVITY_TREE : 4084.2122 sec, 55.75%
- - * TREE_WALK : 2679.5213 sec, 65.61%
- - * TREE_BUILD : 468.6477 sec, 11.47%
- - * TREE_FINAL : 163.3697 sec, 4.00%
- - * TREE_WAIT : 161.9258 sec, 3.96%
- - * TREE_PREP_EXPORT : 26.2529 sec, 0.64%
- - * TREE_ZERO : 19.7784 sec, 0.48%
- - * TREE_SEND : 8.9026 sec, 0.22%
- - * TREE_COPY_OUT : 6.9565 sec, 0.17%
- - * TREE_RECV : 3.4579 sec, 0.08%
- - * Unaccounted : 545.3994 sec, 13.35%
- * PM : 1543.8848 sec, 21.08%
- - * PM_2 : 1528.0329 sec, 98.97%
- - * PM_PREP : 15.8511 sec, 1.03%
- * DOMAIN : 1293.7433 sec, 17.66%
- - * DOM_EXCHANGE : 727.1177 sec, 56.20%
- - * PEANO : 437.8140 sec, 33.84%
- - * RECONSTRUCT_TIMEBINS : 35.9431 sec, 2.78%
- - * Unaccounted : 92.8685 sec, 7.18%
- * TIMELINE : 88.7376 sec, 1.21%
- * SECOND_HALF_KICK : 77.0692 sec, 1.05%
- * FIRST_HALF_KICK : 70.7289 sec, 0.97%
- * TREEUPDATE : 67.3999 sec, 0.92%
- * RESTART : 49.6153 sec, 0.68%
- * DRIFT : 34.3552 sec, 0.47%
- * SNAPSHOT : 5.6949 sec, 0.08%
- * HYDRO_ACCEL : 0.0025 sec, 0.00%
- * Unaccounted : 9.9596 sec, 0.14%
```

Timer output

TIMER_WRITE_SYMBOLS_TO_BALANCEFILE();

Writing of “balance.txt” and “balance-head.txt”.

```

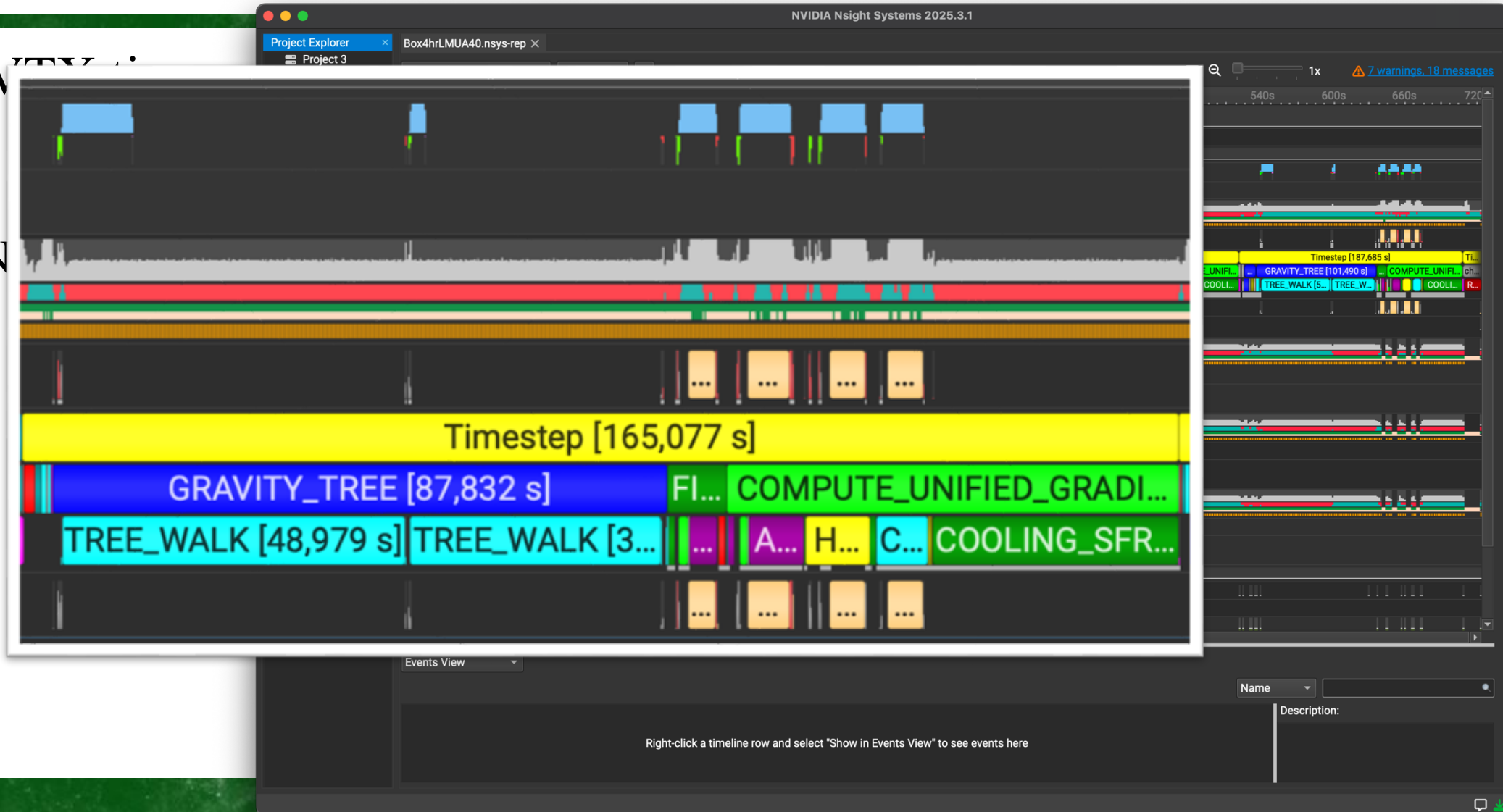
1  Headers for balance file:
2  '0' - DOMAIN
3  '9' - DOMAIN:DOM_EXCHANGE
4  '8' - DOMAIN:PEANO
5  '7' - DOMAIN:RECONSTRUCT_TIMEBINS
6  '6' - DRIFT
7  '5' - FIRST_HALF_KICK
8  '4' - GRAVITY_TREE
9  '3' - GRAVITY_TREE:TREE_BUILD
10 '2' - GRAVITY_TREE:TREE_COPY_OUT
11 '1' - GRAVITY_TREE:TREE_FINAL
12 '}' - GRAVITY_TREE:TREE_PREP_EXPORT
13 ')' - GRAVITY_TREE:TREE_RECV
14 ']' - GRAVITY_TREE:TREE_SEND
15 ''' - GRAVITY_TREE:TREE_WAIT
16 '.' - GRAVITY_TREE:TREE_WALK
17 ';' - GRAVITY_TREE:TREE_ZERO
18 '<' - HYDRO_ACCEL
19 '=' - Initialisation
20 '-' - Initialisation:COMPUTE_UNIFIED_GRADIENTS
21 '(' - Initialisation:DOMAIN
22 '*' - Initialisation:FIND_HSMML
23 '&' - Initialisation:GRAVITY_TREE
24 '^' - Initialisation:HYDRO_ACCEL
25 '%' - Initialisation:NGB_TREEBUILD
26 '$' - Initialisation:PM
27 '@' - Initialisation:RECONSTRUCT_TIMEBINS
28 '#' - Initialisation:reading_ICs
29 '/' - PM
30 '|' - PM:PM_2
31 '[' - PM:PM_PREP
32 '{' - SECOND_HALF_KICK
33 '+' - TIMELINE
34 'Q' - TREEUPDATE
35 '!' - Timestep

```

5

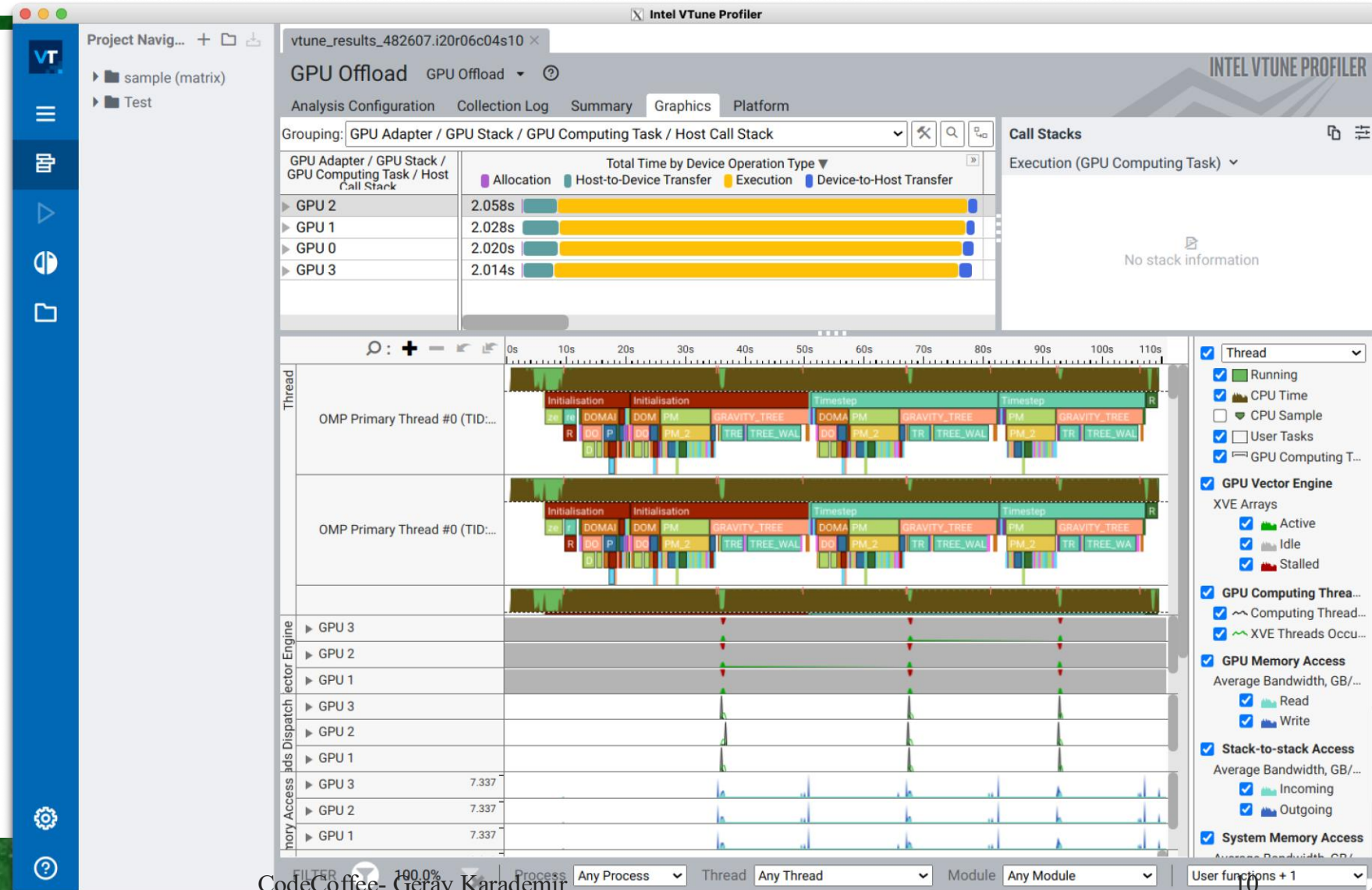
Extensions - NVIDIA

- NVIDIA NVML for use with Systems
- TIMER_N



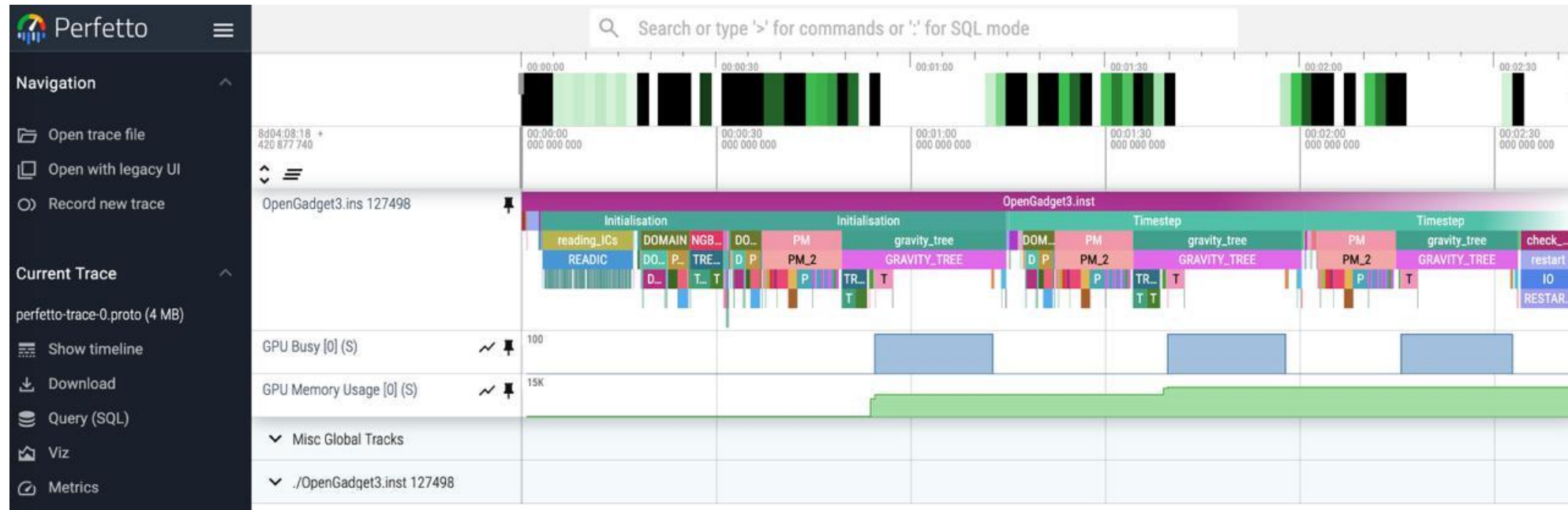
Extensions - INTEL

- ITT Api for use with Intel VTune
 - TIMER_VTUNE



Extensions - AMD

- ROCTx/Omnitrace API for use with Perfetto
 - TIMER_ROCTx or TIMER_OMNITRACE



Extensions - Energy

- Energy measurements with MERIC energy-efficient runtime system
 - TIMER_MERIC

